

VERISPRINT

Proof-of-Work Engine for Software Teams

"Say vs Show" — verified engineering progress, translated for everyone.

Master Specification — v2

Differentiation Strategy · PRP · PRD · Workflows & System Flows

Production Architecture · Data Model · Build Brief for Engineering

Prepared: August 2026 — Revision 2 (incorporates differentiation strategy + new feature set)

Purpose: investor/stakeholder walkthrough deck source, and build specification for engineering.

Confidential — internal planning use.

Table of Contents

- 1. Executive Summary (PRP)
- 2. Differentiation Strategy — Why VeriSprint Is Not a Jellyfish Clone
- 3. Personas & Core Use Cases
- 4. Product Requirements Document (PRD) — Full Feature Set by Phase
- 5. New Advanced Features (Not Previously Discussed)
- 6. Workflows & System Flows
- 7. Production Architecture
- 8. Core Data Model
- 9. Pricing & Packaging
- 10. Build Brief for Engineering (Claude Code Ready)
- 11. Meeting Talking Points

1. Executive Summary (PRP)

VeriSprint is a GitHub-connected platform that reads every commit and pull request in depth, and turns raw code activity into a verified, plain-language account of what was actually built — automatically reconciled against what was claimed in standups, tickets, and status updates. It replaces 'trust me, I did it' with evidence anyone, technical or not, can check in seconds.

The starting wedge is narrow on purpose: verify the gap between claimed and shipped work for small-to-mid engineering teams and agencies. The long-term product is broader — a proof-of-work layer that also onboards new hires, drafts client and investor updates, replaces live standups, and gives agencies a trust-building portal to show clients — all powered by the same underlying "deep code understanding" engine.

One-line pitch

"VeriSprint tells you, in plain English, what your engineering team actually shipped — and proves it."

What changed in this revision

- Added an explicit differentiation strategy so this is not read as a smaller Jellyfish/LinearB clone.
- Added 12 new advanced features not previously discussed in earlier planning (Section 5).
- Added a full data model and an engineering build brief scoped for an AI coding agent (Claude Code) to implement toward a working MVP.
- Reorganized the feature set by build phase (MVP / V2 / V3) so engineering knows exactly what to build first for the upcoming meeting/demo.

2. Differentiation Strategy

Jellyfish, LinearB, and Swarmia already exist and are well-funded. VeriSprint does not win by copying their dashboards smaller and cheaper. It wins by being structurally different on six axes:

Axis	Incumbents (Jellyfish / LinearB / Swarmia)	VeriSprint
What it sells	Metrics & dashboards (cycle time, DORA, investment allocation)	A judgment: is this actually done, with cited evidence
Who it's for	VP of Engineering, enterprise leadership	PM, founder, agency owner, non-technical stakeholder
Onboarding	Sales-led, weeks of setup	Self-serve, GitHub OAuth to first insight in under 5 minutes
Framing	Team performance reporting (feels like surveillance)	Developer-first: saves the developer time, verification is a side effect
Transparency	Black-box scores	Every score shows its evidence: which file, which test, which line
Reach	Internal engineering org only	Extends outward to clients, investors, and new-hire onboarding

The four pillars that make this genuinely yours

- **Evidence-first judgment, not vibes-based scoring.** Every completion score is traceable to specific evidence (tests added, function called elsewhere, error handling present, matches ticket's acceptance criteria). This is inspectable and arguable — which is what makes people trust it instead of dismissing it.
- **Developer-first framing, not manager-first.** The first thing the product does for a developer is save them time (auto-drafted standup, auto-generated documentation). Verification for the PM is a byproduct of a tool developers actually like, not a surveillance layer imposed on them. This flips the adoption dynamics that kill incumbent tools.
- **Outward-facing, not just internal.** Incumbents stop at the engineering org's walls. VeriSprint's same "translate code into plain truth" engine can face clients (agency proof-of-work portal), investors (auto progress updates), and new hires (auto-generated onboarding docs) — three revenue-relevant audiences competitors never touch.
- **Narrow, fast, self-serve.** No sales call, no multi-week rollout. Connect GitHub, see your first verified report same day. This targets the buyer incumbents ignore: the 5–50 person team that can't justify a \$100K enterprise contract or a multi-week implementation.

3. Personas & Core Use Cases

Persona	Core need	Primary feature they live in
Non-technical founder	Know if engineering progress is real before the next investor update	Investor Update Generator, Confidence-Weighted Burndown
Product manager	Verify standup claims without reading code	Claimed vs Shipped, Mismatch Alerts, AI Repo Chat
Agency owner	Prove billed hours match real delivered work to clients	Client Proof-of-Work Portal, White-labeled reports
Engineering manager	Spot stuck/blocked work early without micromanaging	Blocker Detection, Ticket Drift Detector
Developer	Stop re-explaining work every day; get credit for real progress	Auto-Drafted Standup, Personal Evidence Ledger
New hire	Understand an unfamiliar codebase fast	Auto-Generated Onboarding Docs

4. Product Requirements Document (PRD) — Full Feature Set by Phase

Every feature discussed across this planning process, consolidated and phased. This table is the source of truth for what gets built first.

Phase 1 — MVP (build for the demo/meeting)

Feature	What it does
GitHub connection	OAuth + GitHub App install, read-only scopes, repo selection.
Deep commit/PR analysis	LLM reads the diff in file context and explains the functional change in plain English.
Evidence Ledger	Every AI judgment shows its cited evidence: file, line range, test result, TODO/stub detection.
Completion-confidence score	Score per ticket/PR grounded in the Evidence Ledger, not raw line counts.
Auto-drafted standup update	Generates the developer's own 'what I actually did' summary from real commits, editable before posting.
Claimed vs Shipped view	Side-by-side comparison of standup/ticket claims against real commit evidence.
Mismatch alerts	Flags (as questions, not accusations) when a ticket has no matching commits or vice versa.
PM dashboard	Single view: per-person, per-ticket confidence and status.
Developer view	Same underlying data, framed as personal progress and time-saved, not a report card.

Phase 2 — Early paid tier

Feature	What it does
AI Repo Chat	Natural-language Q&A over commit history: "did we ship Stripe integration this week?"
Sprint rollup report	Auto-generated, stakeholder-facing summary of what actually shipped vs what was planned.
Slack/email digests	Daily and sprint-end summaries delivered where the team already works.
Historical accuracy score	Per person, over time: how often claimed progress matched shipped code — framed as calibration, not punishment.
Ticket Drift Detector	Flags when a ticket's real scope silently expanded/changed without re-estimation.

Feature	What it does
Orphan Commit Detector	Finds real work done with no linked ticket, and suggests retroactively creating one for accurate tracking.
Cross-file impact map	Simple visual of which parts of the product (not code) a change touches.

Phase 3 — Growth / expansion

Feature	What it does
Client Proof-of-Work Portal	White-labeled, agency-branded view clients can check directly — no login sharing needed.
Investor Update Generator	Auto-drafts a monthly 'what we built' section for investor updates from verified commit history.
Confidence-Weighted Burndown	Real burndown chart based on verified completion, not self-reported percentage.
Onboarding Doc Generator	Turns the codebase's real structure and recent history into living onboarding documentation for new hires.
Async Standup Replacement	Fully replaces live standup meetings with a verified async report; tracks meeting-time saved.
ROI / Time-Saved Calculator	Quantifies hours of standup/status-meeting time saved in dollar terms, for renewal conversations.
Anomaly check-in nudges	Gently flags unusual activity drops (e.g., no commits in days) as a suggested check-in, not a penalty.
Multi-repo / monorepo intelligence	Understands work that spans multiple repos or services as one logical unit of work.
Private/On-prem LLM option	For security-sensitive customers who can't send code to a third-party API.
Compliance/audit trail mode	Exportable, timestamped record tying code changes to reviews and tickets, for regulated industries.

5. New Advanced Features (Not Previously Discussed)

These are genuinely new ideas beyond everything covered earlier in this planning process. They are what push VeriSprint from 'a cheaper metrics tool' to a product with its own identity.

1 **AI Repo Chat ("Ask Your Codebase")**

A chat interface, in Slack or the dashboard, where a PM types a plain question — "did we finish the checkout redesign this week?" — and gets an answer grounded in real commits and PRs, with citations. This turns VeriSprint from a dashboard you check into an assistant you ask, which is a fundamentally different (and stickier) product interaction.

2 **Client Proof-of-Work Portal (white-labeled)**

For agencies and freelancers: a branded, read-only portal clients can check any time to see verified proof of what was delivered against what was billed. This directly monetizes trust with an audience (clients) that no competitor currently serves, and is a natural high-margin add-on for the agency vertical.

3 **Investor Update Generator**

For founders: auto-drafts the 'what we built' section of a monthly investor update, sourced from verified commit history rather than a founder's memory. Turns a painful monthly chore into a one-click export, and is a strong wedge into the founder/non-technical buyer persona specifically.

4 **Async Standup Replacement + ROI Calculator**

Once claimed-vs-shipped confidence is high enough, VeriSprint can fully replace the live standup meeting with a verified async report, and quantify the hours of meeting time saved across the team in dollar terms. This gives sales and renewal conversations a hard, defensible ROI number instead of a vague productivity claim.

5 **Ticket Drift Detector**

Flags when a ticket's real scope has silently grown or changed based on the code, without a corresponding re-estimation or ticket update — a common, invisible cause of blown timelines that no current tool detects automatically.

6 **Orphan Commit Detector**

Finds genuine work that was done but never linked to any ticket, and offers to retroactively create one. This closes a common gap where real effort goes untracked and uncredited, especially for fast-moving developers who skip ticket hygiene.

7 **Confidence-Weighted Burndown Chart**

A sprint burndown built from verified completion evidence instead of self-reported percentages, giving PMs a materially more honest read on whether a sprint is actually on track.

8

Auto-Generated Onboarding Documentation

Repurposes the same deep-code-understanding engine to generate living onboarding docs for new hires — how the codebase is structured, what changed recently and why — turning an internal analysis engine into a second, distinct value stream (faster ramp time for new engineers).

9

Anomaly Check-In Nudges

Detects unusual drops in a person's activity pattern and gently suggests a check-in to the manager, framed around wellbeing/blockers rather than performance — a deliberately humane feature that also functions as an early warning system for burnout or being stuck.

10

Multi-Repo / Monorepo Intelligence

Understands that a single feature or task often spans multiple repositories or microservices, and stitches related commits across repos into one logical unit of work rather than reporting on repos in isolation — something most competitors handle poorly.

11

Private / On-Prem LLM Option

For security-conscious or regulated customers, an option to run analysis through a private or on-prem model rather than a shared third-party API, removing a common enterprise sales blocker.

12

Compliance & Audit Trail Mode

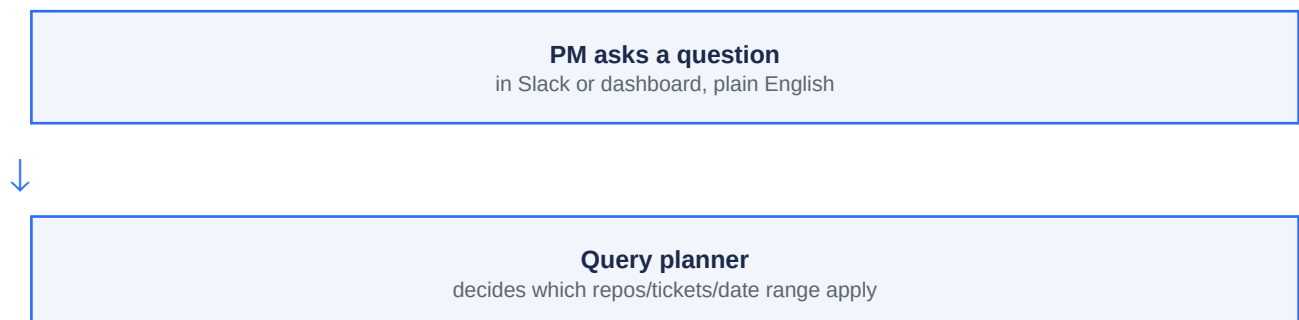
Generates an exportable, timestamped record tying every code change to its review and ticket history — useful for regulated industries (fintech, healthtech) that need to prove process compliance, opening an enterprise upsell path beyond the core PM audience.

6. Workflows & System Flows

Flow A — Core Verification Loop (MVP)



Flow B — AI Repo Chat





Retrieval over Evidence Ledger + vector store



Answer with citations
links back to the specific commits/PRs used

Flow C — Client Proof-of-Work Portal (Agency use case)

Agency completes sprint work



VeriSprint generates a verified summary
plain language, no raw code shown to client



Published to white-labeled client portal



Client views proof of delivered work anytime
builds trust without exposing the codebase

Flow D — Async Standup Replacement

Each developer's commits auto-summarized overnight



Developer reviews/edits their own draft (2 min, async)

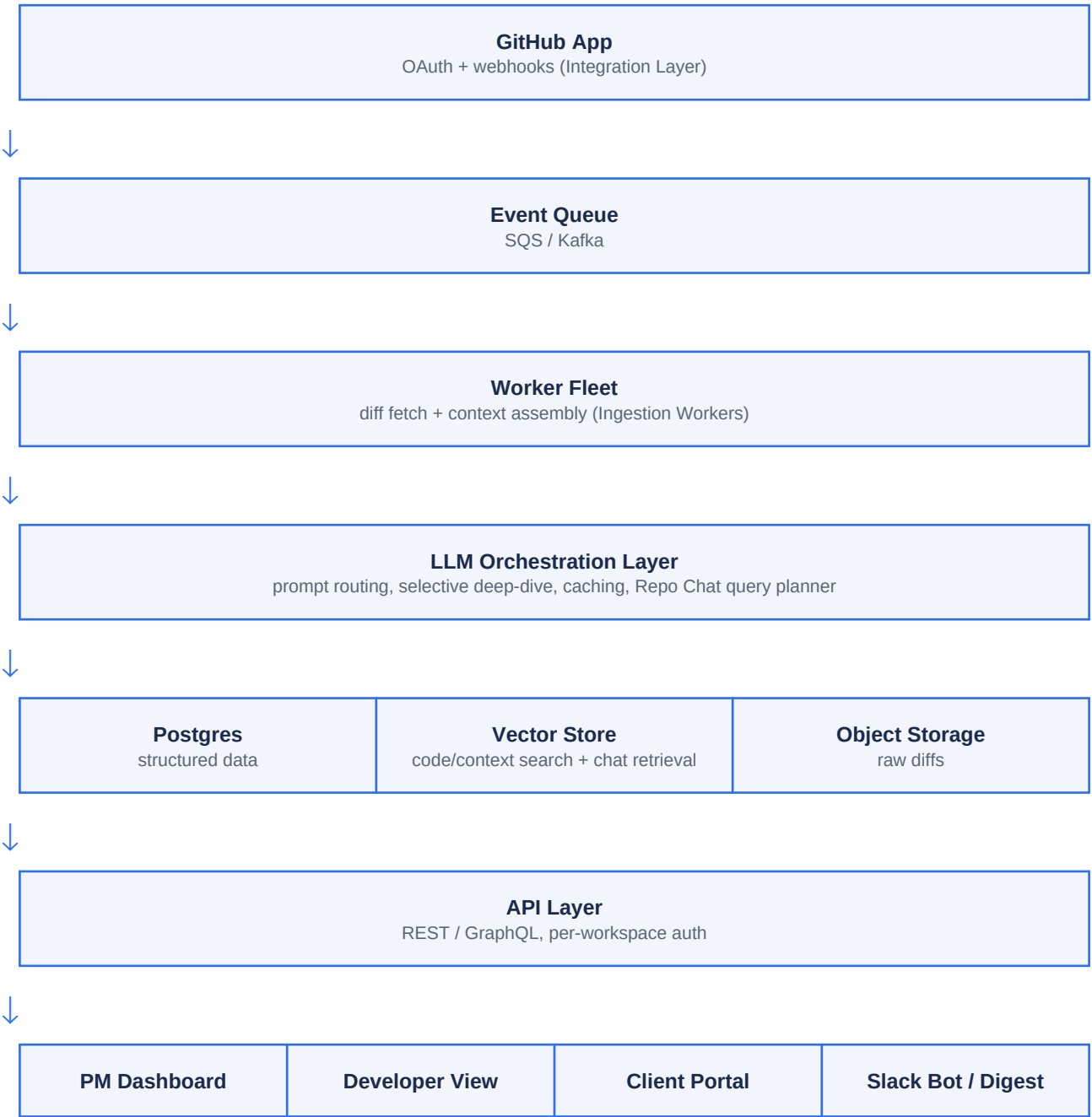


Combined team report posted automatically
replaces the live meeting



Time-saved tracked and shown as ROI over the month

7. Production Architecture



Component Detail

Layer	Technology direction	Purpose
Integration	GitHub App (webhooks + REST/GraphQL API); later GitLab/Bitbucket, Jira/Linear, Slack	Read-only ingestion of commits, PRs, tickets, standup messages.

Layer	Technology direction	Purpose
Queue	Managed queue (SQS/PubSub), Kafka at scale	Decouples ingestion spikes from analysis workers; enables retries.
Ingestion workers	Stateless workers (Node/Python)	Fetch diffs, resolve file context, detect test/TODO/stub signals.
LLM orchestration	Claude via API, tool-calling for selective file deep-dives	Reasons about functional completeness; powers Evidence Ledger and Repo Chat.
Data store	Postgres (structured), object storage (raw diffs), vector store (semantic retrieval)	Persistence, historical scoring, fast retrieval for reconciliation and chat.
Reconciliation engine	Rules + LLM matching service	Aligns commits ↔ tickets ↔ standup text; computes claimed-vs-shipped deltas.
API layer	REST/GraphQL, per-workspace auth	Serves dashboards, client portal, Slack bot, and third-party integrations.
Frontend	React web app (Next.js)	PM dashboard, developer view, agency client portal (white-labeled), settings.
Notifications	Slack app + email digest service	Daily/sprint rollups, mismatch flags, anomaly nudges.
Security	Read-only scopes by default; encryption at rest/in transit; no training on customer code; SOC 2 roadmap	Required to sell into any team with proprietary code, and to unlock the compliance/audit feature.

8. Core Data Model

Minimum entity set an engineering agent needs to implement the MVP end to end.

Entity	Key fields	Notes
Workspace	id, name, plan_tier, github_installation_id	Top-level tenant; one per customer team.
Person	id, workspace_id, github_username, display_name, role	Maps GitHub identity to a human across repos.
Repo	id, workspace_id, github_repo_id, name, default_branch	Connected repositories in scope for analysis.
Commit	id, repo_id, sha, author_id, message, diff_ref, created_at	Raw commit reference; diff content stored in object storage.
PullRequest	id, repo_id, number, title, state, author_id, linked_ticket_id	PR-level grouping of commits.
Ticket	id, workspace_id, external_id (Jira/Linear), title, status, acceptance_criteria	External ticket synced for reconciliation.
EvidenceItem	id, commit_id or pr_id, type (test/todo/stub/call-graph), detail, file_path, line_range	Atomic evidence backing a confidence score — the core trust primitive.
ConfidenceScore	id, ticket_id, score, rationale, evidence_item_ids[], computed_at	Per-ticket completion judgment, always traceable to EvidenceItems.
StandupEntry	id, person_id, workspace_id, text, source (manual/auto-draft), date	Claimed progress, human or AI-drafted.
ReconciliationFlag	id, ticket_id or person_id, type (mismatch/orphan/drift), status, note	Output of the reconciliation engine; drives alerts.
ClientPortalReport	id, workspace_id, period, summary_text, published_at	White-labeled, client-facing rollup (no raw code).
ChatQuery	id, workspace_id, person_id, question, answer, cited_evidence_ids[]	Log of AI Repo Chat interactions, for auditability and improvement.

9. Pricing & Packaging

Tier	Price direction	Includes
Free	\$0	Auto-drafted standup generator for individual developers (GitHub Action / CLI).
Team	\$12–20 / developer / month	Full dashboard, Evidence Ledger, reconciliation, mismatch alerts, Slack digest.
Growth	\$25–35 / developer / month	AI Repo Chat, sprint rollups, ticket drift & orphan detection, historical accuracy.
Agency / Client-Facing	Custom, per portal or flat monthly add-on	White-labeled Client Proof-of-Work Portal, investor update generator.
Enterprise	Custom	Private/on-prem LLM option, compliance/audit trail mode, SSO, multi-repo intelligence.

10. Build Brief for Engineering (Claude Code Ready)

This section is written to be handed directly to an engineering agent (e.g. Claude Code) to scaffold and build toward a demoable MVP.

Recommended stack

- **Frontend:** Next.js + React + Tailwind, deployed on Vercel for fastest iteration.
- **Backend/API:** Node.js (TypeScript) or Python (FastAPI) — pick one consistently for the whole ingestion + API layer.
- **Database:** Postgres (managed, e.g. Supabase/Neon/RDS) for structured data; pgvector or a managed vector DB for retrieval.
- **Queue:** Managed queue (SQS or a hosted Redis-based queue like BullMQ) for ingestion jobs.
- **LLM:** Claude API for diff analysis, Evidence Ledger generation, and Repo Chat query answering.
- **Auth:** GitHub OAuth for login; GitHub App install flow for repo access (read-only scopes only for MVP).
- **Notifications:** Slack app (Bolt SDK) for digests and Repo Chat; Resend/Postmark for email digests.

MVP build order (map to Phase 1 features in Section 4)

- **Step 1:** GitHub App install + OAuth; pull commit/PR history for selected repos into Postgres.
- **Step 2:** Ingestion worker: on webhook, fetch diff + surrounding file context, store raw diff in object storage.
- **Step 3:** LLM analysis pipeline: generate plain-English summary + EvidenceItems (tests/TODOs/dead code/call graph) per commit/PR.
- **Step 4:** ConfidenceScore engine: aggregate EvidenceItems per ticket into a score + rationale.
- **Step 5:** Ticket sync (Jira/Linear API or manual ticket entry for demo) + reconciliation engine producing ReconciliationFlags.
- **Step 6:** Auto-drafted standup generator (reuses Step 3 output, filtered to one person/one day).
- **Step 7:** PM dashboard (claimed vs shipped, confidence scores, flags) and Developer view (same data, personal framing).
- **Step 8:** Slack digest delivery for daily/sprint summaries.

Definition of done for the meeting/demo

- One real GitHub repo connected live, showing actual analyzed commits — not mock data.
- At least one ticket showing a Confidence Score with visible Evidence Ledger (this is the moment that sells the product).
- One example of a Claimed vs Shipped mismatch flag, shown as a question, not an accusation.
- One auto-drafted standup update generated from real commits.

Environment variables to scaffold

GITHUB_APP_ID, GITHUB_APP_PRIVATE_KEY, GITHUB_WEBHOOK_SECRET, ANTHROPIC_API_KEY, DATABASE_URL, VECTOR_DB_URL, SLACK_BOT_TOKEN, SLACK_SIGNING_SECRET, JIRA_API_TOKEN (optional for MVP), OBJECT_STORAGE_BUCKET.

11. Meeting Talking Points

Suggested narrative order for the walkthrough:

- **Open with the pain:** "I've done this" in standup vs. what actually shipped — a gap every non-technical PM and founder feels but can't currently check.
- **Show the wedge, not the whole platform:** lead with the Evidence Ledger and Confidence Score — this is the single feature that makes people lean in, because it shows its work.
- **Name the incumbents on purpose:** Jellyfish, LinearB, Swarmia exist and are funded — then explain the four pillars (Section 2) that make this a different bet, not a smaller copy.
- **Close with reach beyond engineering:** the same engine drafts investor updates and powers a client-facing proof-of-work portal — two revenue paths competitors don't touch.
- **End with the realistic build timeline:** MVP in 4–8 weeks, first paying customers in 2–3 months via the free-tool funnel, meaningful recurring revenue in 6–12 months — credible, not hype.

End of document.